Option Greeks in One Hardware Pass

An FPGA Engine for Heston-Model Greeks by Adjoint Differentiation

Yash Agrawal

Birla Institute of Technology and Science, Pilani

Course registered under: **Prof. Natasha Hazarika**

Supervisor: **Prof. Govind Prasad**

Project report, 6 October 2026

Abstract

Risk management needs not only option prices but their sensitivities to every input, the Greeks. Under the Heston stochastic-volatility model the price has no closed form, and the standard way to obtain Greeks, bump-and-reprice, costs 19 pricings for nine Greeks. This project builds adjoint (reverse-mode) differentiation as a hardware datapath. The Heston price, computed by the COS Fourier-cosine method, and its adjoint are written once as an operation graph; a generator turns it into a fixed-point, resource-shared, modulo-scheduled Verilog engine, together with a bit-accurate model and a first-order bound on the rounding error of every output. The engine returns the price and all nine Greeks in 4,733 clock cycles, only 1.04 times the cost of the price alone and 18.4 times fewer cycles than bump-and-reprice on the same hardware. It fits a Zynq-7020 FPGA; with setup and finish on the host, its loop takes 65.3 μs per evaluation on a ZedBoard and produced results bit-identical to the model on 50 random cases in silicon. Hand-derived analytic Greeks on one laptop CPU core are faster than that (20.3 μs). The 64-bit configuration, run bit-exact on a ZCU102 (101 evaluations), takes 15.9 μs for the whole evaluation: about 1.3 times faster than analytic Greeks on one core and 4.9–15 times faster than forward-mode, adjoint and bump-and-reprice software. The engine's main gains are the cost of its Greeks and a rounding-error bound on each. Forward-mode datapaths built by the same generator need more multipliers for the same Greeks: 25% more when hand-factored like analytic Greeks, 3.2 times more as an AD tool would build them. A small case study on a year of real NIFTY option data illustrates the Greeks in use for hedging.

Contents

1 Introduction **3**

1.1 The problem . . . 3

1.2 What this project does . . . 3

1.3 Novelty . . . 3

2 System architecture **4**

2.1 Design flow . . . 4

2.2 The system on the board . . . 5

2.3 Inside the engine . . . 5

3 Results **6**

3.1 Speed: clock cycles for the price and nine Greeks . . . 7

3.2 Design space: cycles against hardware size . . . 8

3.3 Adjoint against forward mode on the same hardware . . . 9

3.4 Accuracy and the first-order error bound 10
3.5 COS truncation error: a separate term 12
3.6 Accuracy against bump-and-reprice 13
3.7 Accuracy against word length 13
3.8 Calls through put-call parity 14
3.9 Implementation on the Zynq-7020 14
3.10 On silicon 15
3.11 The 64-bit engine on a Zynq UltraScale+ (ZCU102) 16
3.12 Power from simulated switching activity 16
3.13 Against a CPU 17
3.14 Many strikes at once (design study) 18
3.15 Illustration: hedging on real market data 18
3.16 Improvement over the first generation 21

4 Timeline **21**

1 Introduction

1.1 The problem

A bank holding options must know, continuously, how each price responds to the spot price, strike, maturity, interest rate and every parameter of its pricing model. These first derivatives, the *Greeks*, drive hedging and risk limits. The Heston model [1] lets volatility follow its own random process, which reproduces the volatility smile that constant-volatility models cannot. Its price has no closed form and is computed numerically; this project uses the COS method [2], which expands the payoff in a 128-term cosine series and needs only the model's characteristic function.

Two ways exist to obtain Greeks from such a pricer:

- **Bump-and-reprice** nudges each input and prices again. Nine Greeks by central differences cost 19 pricings, and the result depends on a bump size with no safe choice: too small and rounding dominates, too large and truncation does.
- **Adjoint differentiation** (reverse mode, often called AAD in finance [3, 4]) obtains every Greek from one backward sweep over the pricing computation, exactly and at a small constant multiple of one pricing. In software it records a tape of every operation and replays it backwards, which costs 3–7 pricings in practice [5].

Adjoint differentiation is standard in software. On FPGAs, every published Greeks engine still re-runs the pricer once per sensitivity [7, 9].

Three separate layers are involved, and the project uses all three: the **Heston model** describes how the stock and its volatility move; the **COS method** computes the option price from that model; the **adjoint method** computes the Greeks by differentiating the COS computation. The Greeks are computed exactly from the model, not predicted.

1.2 What this project does

The Heston-COS price and its adjoint sweep are written once as an operation graph. A generator, written for this project, produces from that single description:

- a bit-accurate fixed-point model of the hardware (the golden reference);
- a double-precision model of the same algorithm;
- a first-order bound on the rounding error of the price and each Greek (the COS method's own truncation error is a separate term, reported in Section 3.5);
- a modulo-scheduled Verilog datapath that shares a small number of multipliers and CORDIC units across all operations.

The engine was implemented with Vivado on a Zynq-7020, run on a ZedBoard, and compared against bump-and-reprice on the same hardware and against software on a laptop CPU.

1.3 Novelty

1. **The first FPGA implementation of adjoint (reverse-mode) differentiation for option Greeks.** No published FPGA Greeks engine uses adjoints [7, 8, 9]: the AMD library re-runs the pricer once per sensitivity; the 2024 survey of 99 FPGA option-pricing studies [10] and the 2024 review of fifteen years of adjoint differentiation in finance [6] contain no FPGA adjoint implementation.
2. **The first FPGA implementation of the COS method.** COS pricing had run on CPUs and GPUs [11]; the only FPGA Fourier pricer [9] uses trapezoidal integration and returns the

price alone.

3. **A tape-free adjoint datapath.** The reverse sweep is compiled into a statically scheduled fixed-point datapath that shares its arithmetic units with the forward pass: no tape memory and no replay logic. This is possible because the COS pricer performs the same operations for every input.
4. **All nine Greeks for 1.04 times the cost of one price.** On the same hardware the price alone takes 4,568 cycles and the price with all nine Greeks 4,733. On the measured CPU, Greeks derived by hand cost 1.5 pricings, AD tools 5.7–7.6, and bump-and-reprice 19.
5. **A first-order rounding-error bound for every Greek of a fixed-point adjoint datapath.** Measured error never exceeds 0.103 of the bound over 21 test cases, 0.378 over 10,500 inputs including hard regimes, and the bound held on silicon.
6. **Bit-exact Greeks on silicon.** On a ZedBoard (Zynq-7020), 450 of 450 results from 50 random cases matched the model bit for bit.

Prior work these points rest on. The novelty lies in the combination and its realisation in hardware. Reverse-mode differentiation itself is established [12, 3]; the error bound applies first-order rounding analysis [12] to a datapath that itself performs the adjoint; the arithmetic (Newton-Raphson reciprocals, Tang's exponential, CORDIC) is standard. The literature check behind the novelty points was made on 29 September 2026 and is recorded in the repository (`docs/novelty_assessment.md`).

2 System architecture

2.1 Design flow

Figure 1 shows how one description becomes every artifact. Because the RTL and the golden model come from the same graph, the testbench checks every register at the cycle it becomes valid (1,213 to 1,962 checks per design), not just the outputs, and any property shown for the model holds for the hardware.

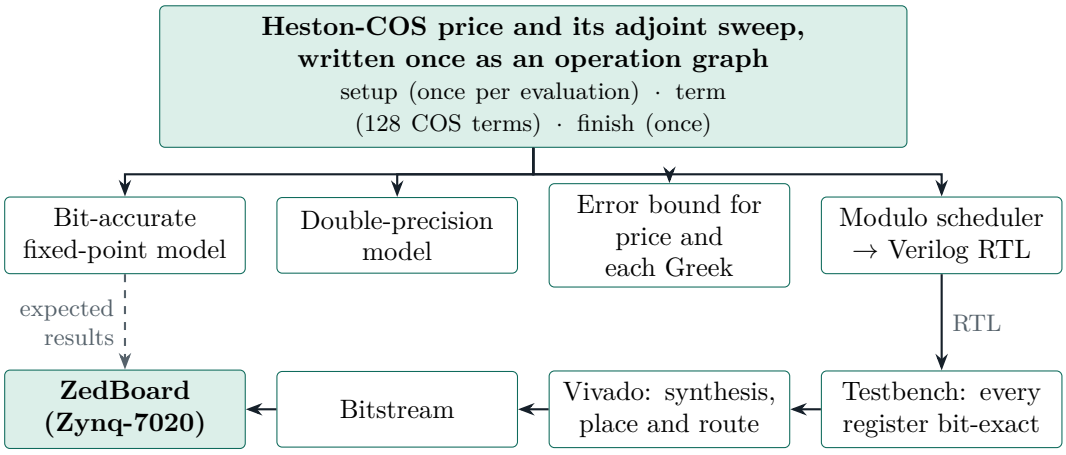


Figure 1: Design flow. The generator derives four artifacts from one operation graph; the RTL is checked register by register against the fixed-point model, implemented in Vivado and run on the ZedBoard, whose results are compared bit for bit with the same model.

2.2 The system on the board

For the board, the once-per-evaluation *setup* and *finish* steps run on the host and the 128-term loop runs on the FPGA. The host writes the nine model inputs and 16 precomputed constants into an AXI4-Lite register file, starts the engine, polls for completion and reads nine sums, from which the finish step forms the price and the nine Greeks (Figure 2). Calls are priced as puts plus put-call parity in the finish step, which removes a cancellation that costs accuracy in fixed point (Section 3.8).

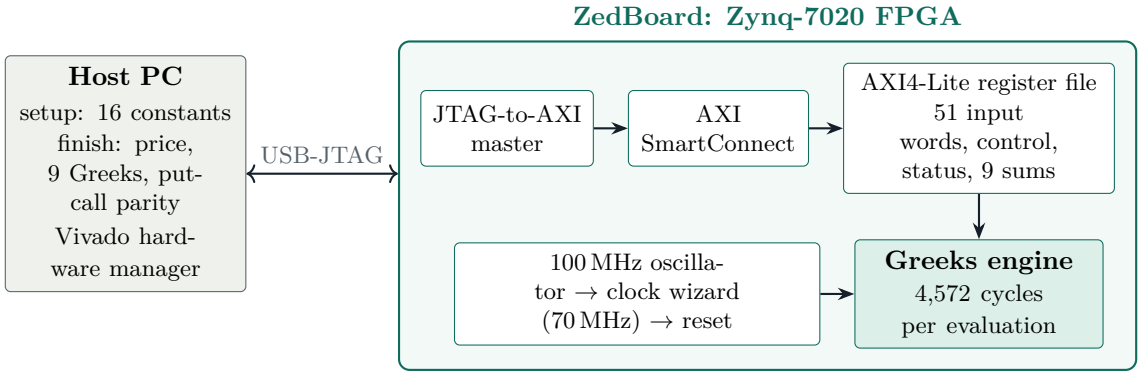


Figure 2: The system built on the ZedBoard (Vivado block design). The engine is driven over the JTAG cable, so the test exercises the datapath alone, with no processor software in the loop.

2.3 Inside the engine

Arithmetic. The engine has no dividers. Reciprocals and square roots use Newton-Raphson iterations seeded from small tables, the exponential uses Tang's method, the logarithm a table and a short series, and sine and cosine a CORDIC unit. As a result every COS term costs exactly 250 multiplications, 3 CORDIC rotations and 2 CORDIC vectorings, and all of it runs on **8 shared 56-bit multipliers** and **5 CORDIC units**.

Per term (Figure 3), the forward pass evaluates the payoff coefficient and the characteristic function; the reverse sweep then propagates adjoints back to the model inputs, reusing the forward pass's reciprocals; a power-of-two normalisation keeps the small adjoints of high-frequency terms at full precision; and nine accumulators sum the price and Greek contributions.

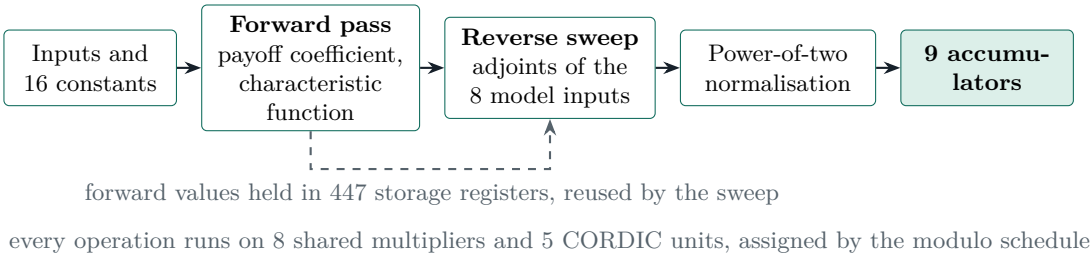


Figure 3: One COS term inside the engine. The adjoint sweep is part of the same datapath as the forward pass; there is no tape memory.

Schedule. The 128 terms are independent, so a new term starts every *initiation interval* of 32 cycles while earlier terms are still in flight (Figure 4). At 8 multipliers, 3 rotation and 2 vectoring units, all three resource classes give the same interval: $\lceil 250/8 \rceil = 32$ and $3 \times 32/3 = 2 \times 32/2 = 32$. One evaluation takes $127 \times 32 + 506 + 2 = 4{,}572$ cycles on the FPGA. Because the CORDIC units set the pace, the forward pass leaves multiplier slots idle in every 32-cycle window; the reverse sweep's multiplications fill them, which is why the nine Greeks cost almost nothing.

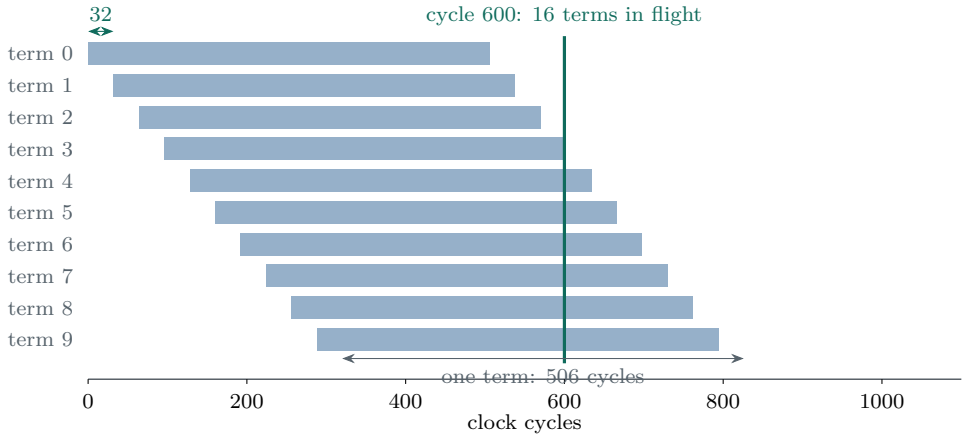


Figure 4: The modulo schedule (first ten of 128 terms). A term lasts 506 cycles and a new one starts every 32, so about 16 terms share the arithmetic units at any time.

Precision. The datapath uses 56-bit fixed-point words with 28 fractional bits, rounding to nearest at every step. An FPGA has no fixed word length: adders and multipliers are built at whatever width the design needs, and a 56×56-bit multiplier is assembled from nine of the chip's 25×18-bit DSP blocks (72 DSP blocks for the 8 multipliers). Section 3.7 shows why 56 bits. Data-dependent shift amounts were measured over the valid input range and the shifters built only that wide, which cut logic by 35%; any input that would exceed a measured range raises a `range_err` flag instead of returning a wrong value.

3 Results

Table 1 summarises the results; each is detailed below with why it is useful and what is new.

Measure	Result	Compared with
Cycles, price + 9 Greeks	4,733	92× fewer than the first-generation engine; 18.4× fewer than bump-and-reprice
Cost of the 9 Greeks	1.04 pricings	hand-derived software 1.5; AD tools 5.7–7.6; bump-and-reprice 19
Latency on the ZedBoard (loop only)	65.3 μs (57.8 μs at 79 MHz)	1.2–3.6× faster than AD tools and bumping on one CPU core; 3.2× slower than hand-derived Greeks (20.3 μs)
Latency, 64-bit on a ZCU102	15.9 μs, whole evaluation (100 MHz; 101 board evaluations bit-exact)	about 1.3× faster than hand-derived Greeks on one core; 4.9–15× faster than AD tools and bumping
Energy per evaluation	44 μJ (ZedBoard loop), 40 μJ (64-bit, whole evaluation); engine power from simulated activity	about 2–4× less than the CPU at full throughput (CPU power estimated)
Accuracy (21 cases)	price 6.1×10^{-7}, Greeks $\leq 1.2 \times 10^{-5}$	relative to double precision; over 10,500 inputs worst absolute 1.2×10^{-3} ($\partial V/\partial \xi$)
Error bound	never exceeded (≤ 0.378)	every output, 10,500 inputs
On silicon	450 / 450 bit-exact	50 random cases
Area	73% LUT, 72 of 220 DSP	fits a Zynq-7020; the first generation did not

Table 1: Results at a glance (Zynq-7020 configuration).

3.1 Speed: clock cycles for the price and nine Greeks

Design	Price + 9 Greeks	Price only	Bump-and-reprice	Bump ÷ adjoint	Greeks cost
First generation (hand-written, 64-bit)	433,779	200,562	3,811,647	8.8×	2.16×
Generated, Zynq-7020 (56-bit, 8 multipliers)	**4,733**	4,568	86,860	**18.4**×	**1.04**×
Generated, board design (loop on FPGA)	4,572	–	–	–	–
Generated, 64-bit (32 multipliers)	1,593	1,026	19,562	12.3×	1.55×

Table 2: Clock cycles per evaluation, from cycle-exact RTL simulation (128 COS terms). "Greeks cost" is the price-with-Greeks cycles divided by the price-only cycles on the same hardware.

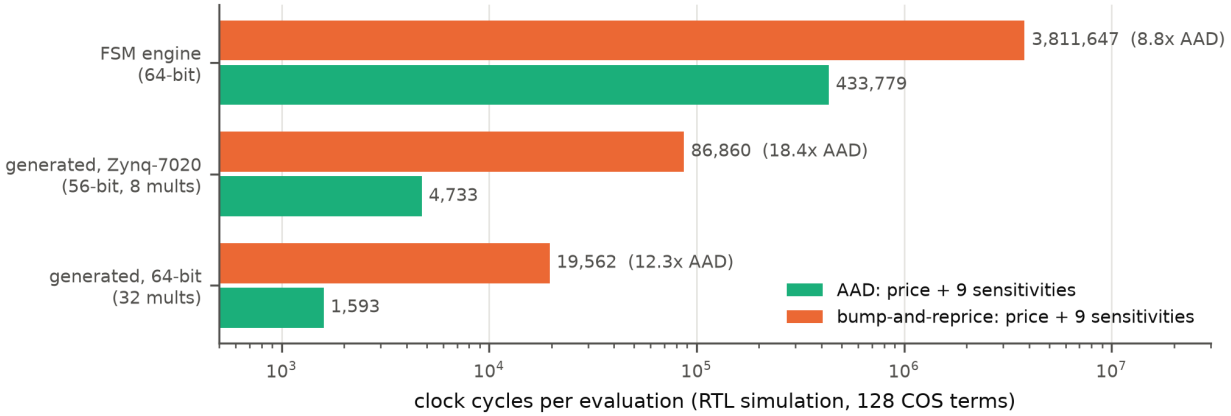


Figure 5: Cycles per evaluation, adjoint against bump-and-reprice on the same hardware (log scale).

Why it is useful. A full risk report for one option, price and nine Greeks, costs only 3.6% more than its price alone: 165 extra cycles. Risk can be recomputed as often as prices, instead of 18.4 times less often under bump-and-reprice.

What is new. No hardware had computed Greeks by adjoint differentiation before, so no hardware had shown this cost. Software adjoint differentiation costs 5.3–6.7 pricings because of its tape; here the reverse sweep fills idle multiplier slots left by the CORDIC-bound forward pass. A forward-mode sweep could fill them too, but needs more multiplies (Section 3.3). On the 64-bit design, where multipliers are the bottleneck, the cost is 1.55 pricings, which is still well below software.

3.2 Design space: cycles against hardware size

Configuration	Multipliers	Interval	Price + Greeks	Price only	Bump	Bump ÷ adjoint
Zynq-7020 (56-bit)	2	125	16,754	8,700	165,368	9.9
Zynq-7020 (56-bit)	4	63	8,694	4,613	87,715	10.1
Zynq-7020 (56-bit)	8	32	4,733	4,568	86,860	18.4
Zynq-7020 (56-bit)	16	32	4,621	4,563	86,765	18.8
64-bit	1	250	32,530	16,773	318,755	9.8
64-bit	4	63	8,700	4,636	88,152	10.1
64-bit	8	32	4,709	2,573	48,955	10.4
64-bit	16	16	2,614	1,540	29,328	11.2
64-bit	32	8	1,593	1,026	19,562	12.3
64-bit	64	4	1,073	888	16,940	15.8
64-bit	128	3	942	888	16,940	18.0

Table 3: Cycles per evaluation as the number of shared multipliers changes. The interval is the number of cycles between the starts of consecutive COS terms.

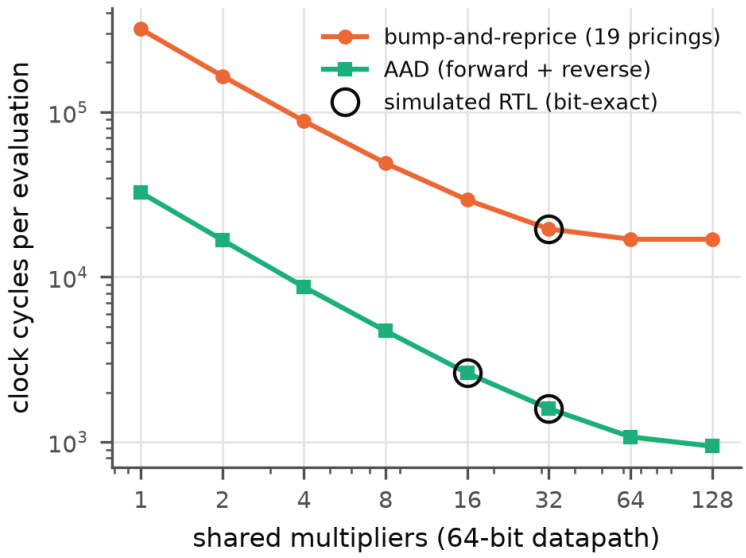


Figure 6: Cycles against shared multipliers (64-bit). Circled points were simulated as RTL and matched the schedule exactly.

Why it is useful. The same generator produces a small engine for a low-cost FPGA or a fast one for a large FPGA from one parameter. Cycles halve as multipliers double until another resource sets the pace; at 8 multipliers on the Zynq-7020 design the CORDIC units do, so a 16th multiplier gains only 2%. The table shows where extra hardware stops paying.

3.3 Adjoint against forward mode on the same hardware

The reverse sweep is not the only way to get all nine Greeks in one evaluation. Three forward-mode (tangent) datapaths were built by the same generator, from the same primitives, with the same frozen truncation range, hoisting and finish, and scheduled by the same scheduler: *dense*, which carries all eight input directions through every operation as a vector forward-mode AD tool does; *sparse*, which skips derivatives that are structurally zero; and *factored*, which forms the derivatives of the characteristic-function exponent with respect to its two complex inner quantities and the maturity once per term and combines them for each Greek, as a careful hand derivation of analytic Greeks does (Cui et al. [14]). All three return the same price and Greeks as the adjoint datapath to within 6×10^{-14} in double precision.

		Zynq-7020 (56-bit)		64-bit	
Datapath	**Multiplies per term**	**8 mult.**	**16 mult.**	**16 mult.**	**32 mult.**
Price only	126	4,566	4,561	1,538	1,024
Adjoint (this work)	**250**	**4,731**	4,619	**2,612**	**1,591**
Forward, factored (analytic)	313	5,622	4,571	3,069	1,795
Forward, sparse	396	6,968	4,571	3,690	2,171
Forward, dense (AD tool)	802	13,597	7,150	7,155	3,822

Table 4: Multiplies per COS term and cycles per evaluation for the price and nine Greeks, by datapath and number of shared multipliers (`validation/run_mode_baselines.py`). The forward datapaths have no fixed-point rescaling of their tangents, which a working engine would need, so their counts are lower bounds.

Why it is useful. For the same Greeks, the adjoint needs the fewest multiplies: 124 per term

beyond the price, against 187 for the best hand-factored forward method and 676 for forward mode as an AD tool builds it. With the board's 8 multipliers that is 4,731 cycles against 5,622 and 13,597. The 1.04 pricings of Table 2 are not unique to the adjoint: with 16 multipliers the CORDIC units set the pace for every variant except the dense one, and all reach about 4,570 cycles. What the adjoint saves is hardware: it reaches that pace with 8 multipliers (72 DSP blocks), where the factored forward method needs 10 (90) and the dense one 26 (234, more than the chip has).

3.4 Accuracy and the first-order error bound

Output	Zynq-7020 (56-bit)	64-bit	Worst error ÷ bound
Price	6.1×10^{-7}	2.8×10^{-8}	0.074
Delta $\partial V/\partial S_0$	2.0×10^{-7}	6.9×10^{-9}	0.049
$\partial V/\partial K$	2.5×10^{-7}	6.6×10^{-9}	0.051
$\partial V/\partial T$	3.1×10^{-7}	6.0×10^{-9}	0.093
Rho $\partial V/\partial r$	2.4×10^{-7}	8.9×10^{-9}	0.049
Vega $\partial V/\partial v_0$	6.2×10^{-7}	2.0×10^{-8}	0.103
$\partial V/\partial \kappa$	7.8×10^{-6}	4.1×10^{-7}	0.098
$\partial V/\partial \theta$	7.9×10^{-7}	3.4×10^{-8}	0.102
$\partial V/\partial \xi$	2.7×10^{-6}	2.2×10^{-7}	0.099
$\partial V/\partial \rho$	1.2×10^{-5}	7.4×10^{-7}	0.087

Table 5: Worst relative error over 21 parameter sets (moneyness, maturity, vol-of-vol, correlation; calls and puts) against the double-precision model, and the largest measured error as a fraction of the first-order bound, across both designs.

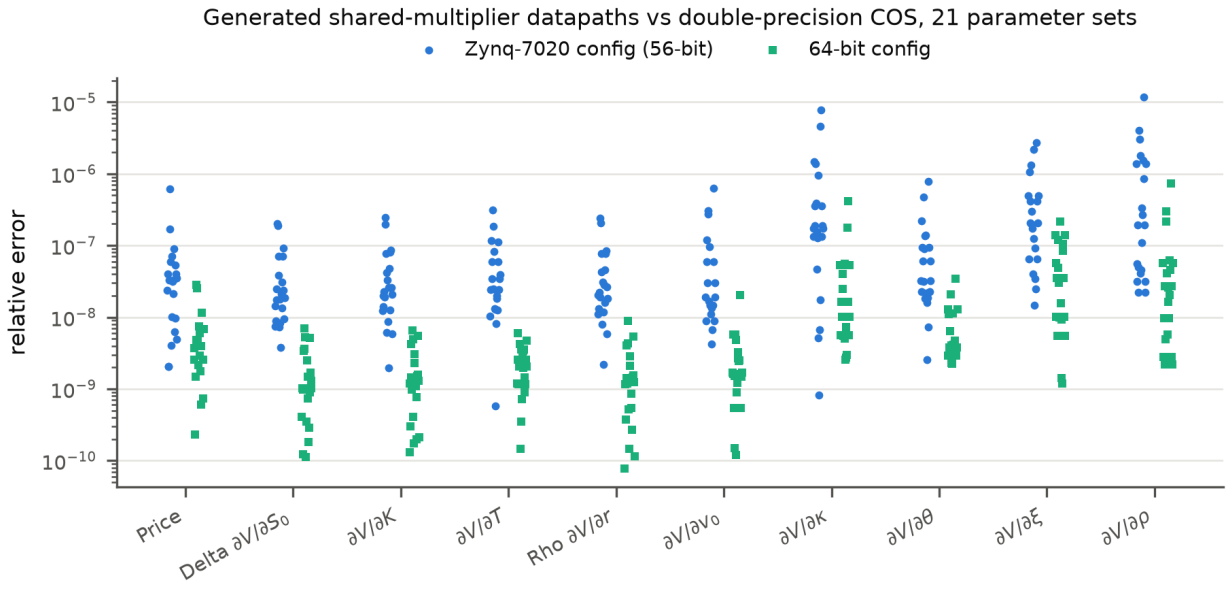


Figure 7: Relative error of every output for both designs over the 21 parameter sets.

A larger sweep tested the 56-bit design on 10,500 inputs: 6,000 uniform over the verified domain, 500 in each of seven hard regimes (Feller condition violated, correlation near its limits, deep strikes, very short and long maturities, high vol-of-vol, very low variance) and 1,000 with one input outside the domain. Each case runs the bit-accurate model, the error bound, the reference, and the same range check that raises the hardware's `range_err` flag.

Output (absolute error)	Median	99th percentile	Worst
Price	3.3×10^{-7}	1.7×10^{-5}	7.3×10^{-5}
Delta	3.5×10^{-9}	1.6×10^{-7}	2.2×10^{-6}
Vega $\partial V/\partial v_0$	2.8×10^{-7}	2.1×10^{-5}	2.9×10^{-4}
$\partial V/\partial \theta$	2.0×10^{-6}	1.1×10^{-4}	3.7×10^{-4}
$\partial V/\partial \xi$	5.9×10^{-7}	2.5×10^{-4}	1.3×10^{-3}

Table 6: 56-bit design, all 9,500 in-domain inputs.

No result exceeded its error bound in any of the 10,482 unflagged cases, inside or outside the domain (worst 0.378 of the bound), with no overflow; the 18 flagged inputs were all outside the domain. The 21-case grid understates the tail: at small vol-of-vol ($\xi \approx 0.1$) with large $\kappa\theta$, $\partial V/\partial \xi$ is off by up to 1.3×10^{-3}, all of it fixed-point rounding and inside its bound. The sweep also exposed false alarms: `range_err` fired on 25 in-domain inputs at very low variance and very short maturity, where two normalization shifts went one step past their fitted range. Fitting the ranges on extra samples at the ten edges of the domain widened 50 of 113 shifters by 1–4 steps and removed every in-domain flag; all designs were regenerated and pass their bit-exact testbenches. Because that sweep had prompted the refit, the refitted ranges were then tested on a fresh, held-out sweep of the same size (seed 20261006, `run_domain_sweep.py -heldout`): none of its 9,500 in-domain inputs raised `range_err`, none exceeded its bound (worst 0.356 of it), and of the 1,000 out-of-domain inputs 9 were flagged and every other one stayed within its bound.

Why it is useful. The error bound is reliable over ten thousand inputs, including the hard regimes: every result the hardware does not flag is within its computed bound, so a user knows each Greek's accuracy before using it, and no valid input is now rejected.

Why it is useful. Against the same algorithm in double precision, over the 9,500 held-out in-domain inputs, the price, delta, strike sensitivity, theta, rho and vega are good to at least 5.3 significant figures in 99% of inputs (median 7.6–8.1; worst 3.3–4.4). The model-parameter sensitivities are weaker: $\partial V/\partial \theta$ and $\partial V/\partial \rho$ give 4.9 figures at the 99th percentile, and $\partial V/\partial \kappa$ and $\partial V/\partial \xi$ only 3.5 and 3.0, falling to 2.2 and 1.4 where they are small (`validation/sigfig_report.py`; inputs where a Greek is within 1% of zero are excluded). Each result comes with a bound computed in 0.1 s per parameter set that the hardware error never exceeded. A risk system can rely on the bound instead of on the cases that happened to be tested.

What is new. FPGA finance work validates fixed point by measuring error over test cases. Here a rounding-error bound is computed for every Greek produced by the adjoint datapath, because the error model includes the hardware's own reverse sweep. Measured error reaches at most 0.103 of the bound.

What the bound covers. The bound is first order: it propagates every rounding error through the derivatives of the computation and drops products of rounding errors, which are smaller by a factor of about 2^{-28} at this word length. It covers fixed-point rounding only, measured against the same algorithm computed exactly; the COS method's own truncation error is a separate term (Section 3.5). Two checks support dropping the second-order terms. First, the same graph was evaluated in 50-digit arithmetic on 100 held-out inputs (`validation/run_mp_check.py`): the hardware's error against the 50-digit value is at most 0.24 of the bound, the same to three decimals as against double precision, and double precision itself is within 5×10^{-5} of the bound from the exact value. Second, the bound still held at 44 bits (Section 3.7), where second-order terms are relatively 4,096 times larger than at 56 bits.

3.5 COS truncation error: a separate term

The engine computes the COS approximation of the Heston price: 128 terms over a truncation range of ±10 standard deviations. Its distance from the exact Heston value is the method's own error, which no word length removes. On the 50 board cases it was measured against an independent Fourier-integral reference (`validation/run_method_error.py`).

Output	Method error, median	Method error, worst	Rounding, worst	Rounding bound, worst
Price	2.1×10^{-11}	4.4×10^{-5}	1.5×10^{-5}	2.8×10^{-4}
Delta	6.9×10^{-10}	8.3×10^{-5}	1.1×10^{-7}	1.6×10^{-6}
Vega	1.3×10^{-9}	7.8×10^{-3}	1.4×10^{-5}	2.0×10^{-4}
$\partial V/\partial\theta$	5.6×10^{-9}	2.5×10^{-3}	7.0×10^{-5}	1.8×10^{-3}
$\partial V/\partial\xi$	1.3×10^{-9}	1.8×10^{-3}	2.1×10^{-4}	4.5×10^{-3}

Table 7: Absolute errors over the 50 board cases. Method error: COS (128 terms, double precision) against the Fourier integral. Rounding: the board's results against COS in double precision. The other outputs behave alike.

In a typical case the method error is far below the rounding error, but in a few it is much larger. The five worst cases have two causes. In three, 128 terms are too few (high volatility of volatility with short maturity: $\xi = 1$, $T = 0.34$ gives the worst vega error); with 256 terms COS matches the integral. In two, the truncation range is too narrow (a deep out-of-the-money put); a range 1.5 times wider matches the integral. The total error of a result against the Heston model is at most the rounding bound plus the method error, and only the first is bounded.

Why it is useful. Separating the two terms shows where accuracy is lost. Rounding is bounded and small; the method error comes from the choice of 128 terms and the range rule, which any implementation of this COS configuration shares, in hardware or software. Where it matters, more terms (256 roughly doubles the cycles) or a wider range would reduce it.

3.6 Accuracy against bump-and-reprice

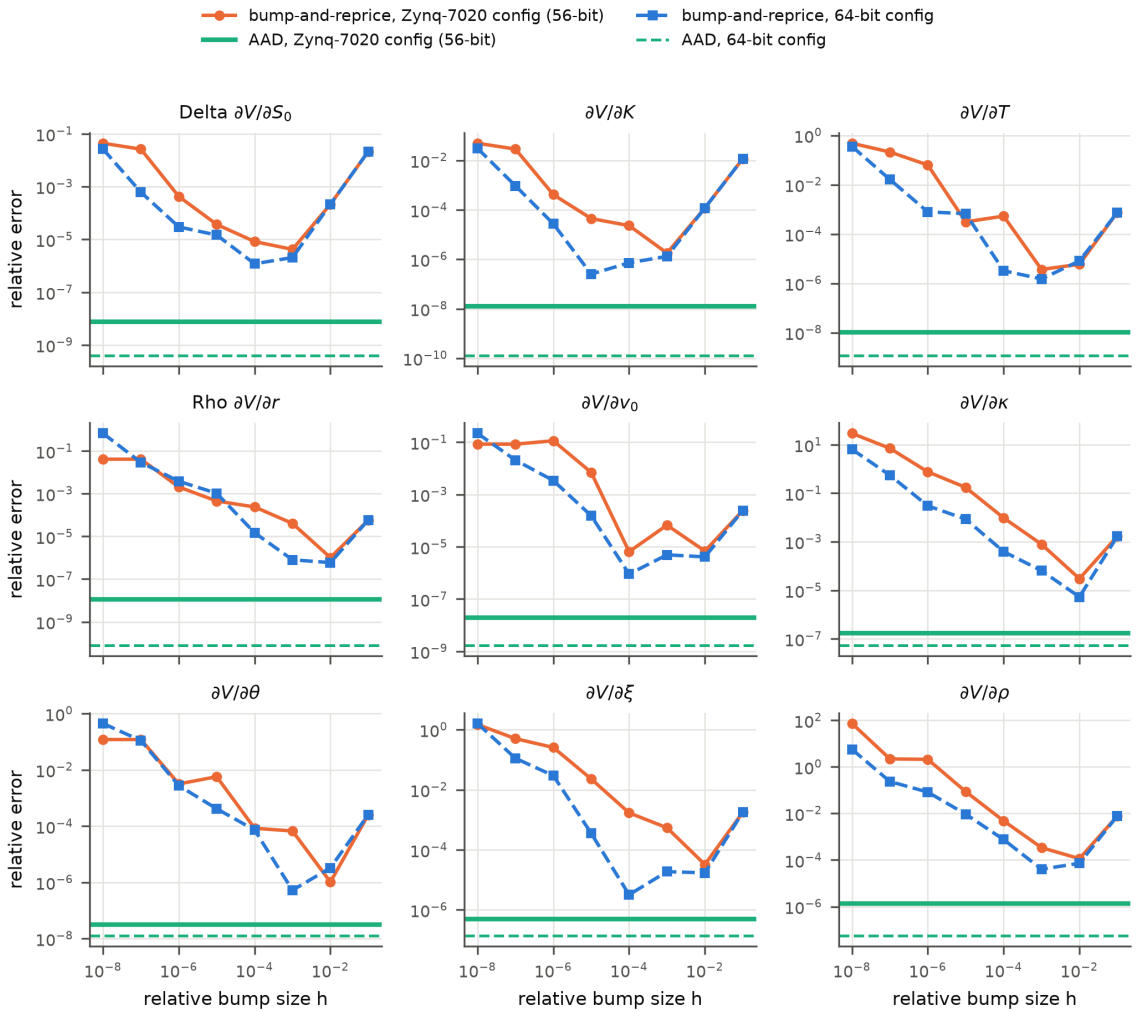


Figure 8: Every Greek: bump-and-reprice error against bump size (curves) and the adjoint engine (flat lines), on the same pricer and word length.

Why it is useful. Bump-and-reprice error is U-shaped in the bump size and has no setting that is safe for every Greek. On the Zynq-7020 design the adjoint Greeks are 32–560 times more accurate than bump-and-reprice even with the best bump size chosen afterwards for each Greek (median 145 times), with nothing to tune.

3.7 Accuracy against word length

Word length (fraction bits)	Worst price error	Worst Greek error	Worst error ÷ bound	Overflows
44-bit (16)	1.5×10^{-3}	2.8×10^{-1}	0.167	0
48-bit (20)	1.8×10^{-4}	9.1×10^{-3}	0.121	0
52-bit (24)	5.0×10^{-6}	5.8×10^{-4}	0.113	0
56-bit (28), this design	6.1×10^{-7}	1.2×10^{-5}	0.103	0
64-bit (32)	2.8×10^{-8}	7.4×10^{-7}	0.099	0
64-bit (36)	1.2×10^{-9}	7.9×10^{-8}	0.150	0

Table 8: Worst relative error over the 21 parameter sets as the word length changes (`validation/results/wordlength_sweep.csv`).

Why it is useful. Every four bits removed costs roughly 10–50 times in Greek accuracy. At 48 bits a Greek can be 1% wrong and at 44 bits 28% wrong, while 64 bits buys accuracy no risk desk needs at a higher hardware cost; at 56 bits the price and the market Greeks are good to at least five significant figures in 99% of inputs, and the model-parameter sensitivities to at least three. The first-order bound held at every width, so it predicts this trade-off without building each design.

3.8 Calls through put-call parity

Priced directly, a call's COS coefficients contain e^b for the upper truncation bound b, which grows large over a wide range and then cancels, losing precision in fixed point. The engine now always prices the put and adds $S_0 - Ke^{-rT}$ and its derivatives for a call.

Worst absolute error, 22 calls	Call priced directly	Put + parity	Improvement
Price	2.3×10^{-3}	3.1×10^{-5}	75×
Price (median)	4.8×10^{-5}	5.4×10^{-7}	≈90×
$\partial V/\partial\kappa$	2.4×10^{-3}	3.9×10^{-4}	6×
$\partial V/\partial\theta$	1.7×10^{-2}	2.5×10^{-3}	7×
$\partial V/\partial\xi$	1.3×10^{-2}	1.8×10^{-3}	7×
$\partial V/\partial\rho$	8.3×10^{-3}	4.7×10^{-4}	18×
Delta, $\partial V/\partial K$, $\partial V/\partial T$, rho, vega			2–2.5×

Table 9: Effect of put-call parity on the 22 calls of the 50-case board set, against the independent Fourier-integral price. It costs 2 cycles (4,731 → 4,733) and removes one multiplication per term.

Why it is useful. The worst call price error falls 75-fold, and every Greek improves. After the change, the hardware's error matches the error of the COS method itself, so fixed-point rounding is no longer the limiting factor.

3.9 Implementation on the Zynq-7020

Build	LUT	FF	DSP	Clock	Slack	Power
Engine alone, 29 Sep design (xc7z020clg400-1)	36,499 (68.6%)	28,559 (26.8%)	72 (32.7%)	100 MHz	−2.639 ns	0.258 W
Engine alone, current design (xc7z020clg484-1)	35,782 (67.3%)	27,204 (25.6%)	72 (32.7%)	70 MHz	+0.109 ns	0.204 W
ZedBoard system (xc7z020clg484-1)	38,752 (72.8%)	32,134 (30.2%)	72 (32.7%)	70 MHz	+0.458 ns	0.340 W

Table 10: Vivado 2025.2 results after routing. The engine alone reaches about 79 MHz; inside the board system the design meets timing at 70 MHz. Power is Vivado's power report (vectorless analysis); 0.121 W of the board figure is the clock generator. With simulated activity the current engine alone draws 0.676 W (Section 3.12).

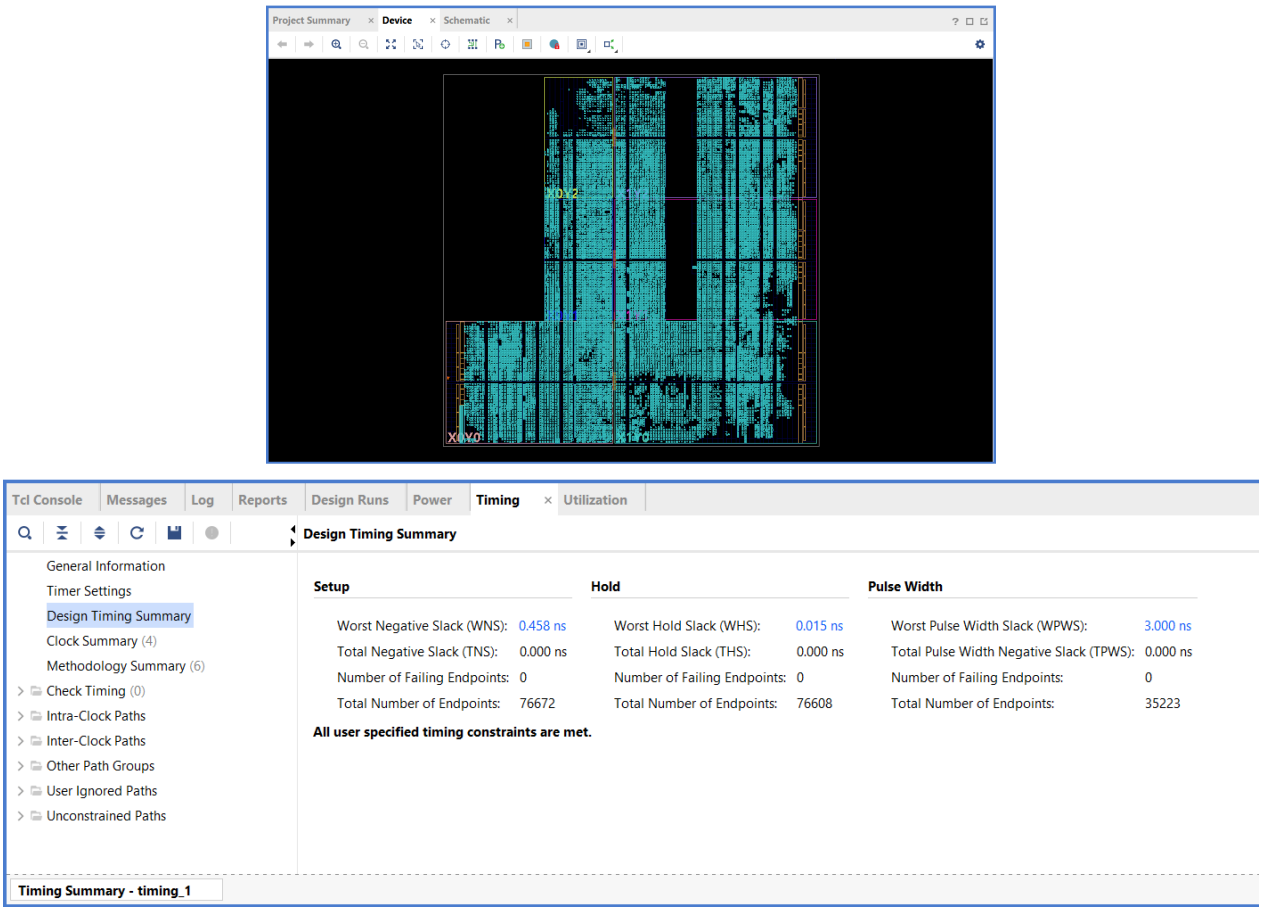


Figure 9: The placed design on the Zynq-7020 (top) and the timing summary of the ZedBoard build of 6 October (put-call parity, refitted ranges): all constraints met at 70 MHz, WNS +0.458 ns, 0 failing endpoints of 76,672 (bottom).

Why it is useful. The engine fits a low-cost Zynq-7020 with room to spare: 73% of the logic and a third of the DSP blocks, leaving space for the processor interface. The first generation could not fit at all.

3.10 On silicon

Run	Cases	Bit-exact results	Range errors	Worst error ÷ bound
First vector	1	9 / 9	0	0.049
Sweep, back to back	50	450 / 450	0	0.199

Table 11: ZedBoard results, 6 October 2026, with the current design (put-call parity and refitted shift ranges). The sweep used 2 reference cases and 48 random cases over the valid input range (22 calls, 28 puts), with no reset between cases. Worst price error over the 50 cases against the double-precision reference: 1.5×10^{-5}, down from 2.3×10^{-3} with the earlier bitstream of 29 September (which was also 450 / 450 bit-exact, worst 0.268 of the bound).

Why it is useful. The hardware computes exactly what the model computes, so every accuracy result and the error bound carry over to the chip. Running the cases back to back also shows that nothing carries over from one evaluation to the next.

What is new. These are the first adjoint-differentiated option Greeks computed on an FPGA.

A second ZedBoard design puts the ARM processor in the loop: the setup and finish steps run on the ARM as C generated from the same operation graph (bit-exact with the model on 300

inputs, with the portable 64×64-bit multiply the 32-bit ARM needs), and the engine sits behind the processor's AXI4-Lite port. It is built (timing met at the processor's 71.4 MHz clock, WNS +0.080 ns) with a program that checks 100 cases bit-exact and times each stage, including the 51-word and 18-word bus transfers that the loop-only latency leaves out; the board measurement is pending.

3.11 The 64-bit engine on a Zynq UltraScale+ (ZCU102)

The 64-bit configuration (32 shared multipliers, setup and finish on chip, 1,593 cycles per evaluation) had only been estimated by Yosys. It was implemented behind the same AXI4-Lite register file and JTAG test system as the ZedBoard, in Vivado 2025.2 for the ZCU104's xczu7ev and in Vivado 2026.1 for the ZCU102's xczu9eg (the part and clock taken from each board's file), and met timing at 100 MHz on both. It was run on the ZCU102: one vector, the 50-case sweep and the 50 real NIFTY days, all ten outputs of all 101 evaluations bit-exact against the model (1,010 of 1,010 words).

	ZCU102	**ZCU104**
Part	xczu9eg-ffvb1156-2-e	xczu7ev-ffvc1156-2-e
LUT	77,515 (28.3%)	77,863 (33.8%)
Registers	63,547 (11.6%)	63,796 (13.8%)
DSP blocks	510 (20.2%)	510 (29.5%)
Slack at 100 MHz	+1.601 ns (Fmax ≈119 MHz)	+1.469 ns (Fmax ≈117 MHz)
Power (vectorless)	1.43 W (0.81 dyn., 0.63 static)	1.20 W (0.60 dyn., 0.60 static)
On the board	**1,010 / 1,010 outputs bit-exact**	not run
Latency, price + 9 Greeks	**15.9 μs**, whole evaluation (1,593 cycles at 100 MHz)	
Engine power, simulated activity	2.49 W on the xczu7ev (1.88 dynamic, 0.60 static)	
Energy per evaluation	40 μJ (engine, simulated activity)	

Table 12: The 64-bit engine on two Zynq UltraScale+ boards (`hardware/vivado/zcu102`, `zcu104`; Yosys had estimated 108,330 LUT). The latency is the whole evaluation, setup and finish included, from the cycle count and clock; it excludes only the 19-word and 10-word bus transfers. The board run checks every output but cannot time the engine, since each JTAG transaction takes far longer than an evaluation.

Why it is useful. On a larger FPGA the same generator gives an engine 4 times faster than the ZedBoard's loop and, narrowly, faster than the best hand-written software: 15.9 μs for the whole evaluation against 20.3 μs for analytic Greeks on one idle CPU core (Table below), about 1.3 times (1.5 times at the 119 MHz the routing allows, not tested on the board). It computes exactly what the model computes on a second FPGA family, and uses about a quarter of the ZCU102's logic, leaving room for more engines (not built).

3.12 Power from simulated switching activity

Vivado's default power report assumes that every signal toggles in 12.5% of cycles. For an engine that is busy every cycle that guess is too low. Each engine was therefore implemented on its own, its routed netlist simulated gate by gate on four evaluations back to back (two board-sweep cases and two real NIFTY days, every output checked bit-exact), and the recorded activity read back into the power analysis (`hardware/vivado/power`).

Engine alone (routed)	Default guess	Simulated activity	Latency	Energy
ZedBoard engine, 70 MHz	0.204 W	**0.676 W**	65.3 μs (loop)	**44 μJ**
64-bit engine (xczu7ev), 100 MHz	1.288 W	**2.485 W**	15.9 μs (whole)	**40 μJ**

Table 13: Engine power with Vivado's default activity and with the activity of the routed netlist simulated on real evaluations (confidence "High"; 57% of nets matched by name, the rest propagated by Vivado). Dynamic power was 0.099 against 0.564 W (ZedBoard) and 0.692 against 1.883 W (64-bit). Engine only: the clock generator, bus and the rest of the board are not included; a board rail measurement is pending. On the ZCU102's xczu9eg the static power is 0.03 W higher (vectorless reports), which leaves the 64-bit engine's energy at 40 μJ.

Why it is useful. Measured activity replaces an assumption that underestimated dynamic power 3 to 6 times. The energy per evaluation is about twice the earlier estimate (44 instead of 22 μJ on the ZedBoard), and the faster 64-bit engine uses slightly less energy per evaluation than the ZedBoard's despite drawing more power.

3.13 Against a CPU

The same algorithm was written in C++ (double precision, g++ 13, -O3 -march=native) and run on an Intel Core i5-1155G7 laptop CPU, with adjoint differentiation by the CoDiPack tool [13]. Every method matches the reference model (adjoint to 1.3×10^{-10} relative).

Price + 9 Greeks	μs per evaluation	Cost vs one price	Evaluations / s
CPU, price only (1 core)	13.7	1×	–
CPU, bump-and-reprice (1 core)	238.1	17.4×	–
CPU, adjoint with CoDiPack (1 core)	103.8	7.6×	26,900 (4 cores)
CPU, forward mode, 9 directions (1 core)	78.0	5.7×	34,000 (4 cores)
CPU, optimised bump* (1 core)	146.6	10.7×	–
CPU, analytic Greeks* (1 core)	**20.3**	1.5×	158,000 (4 cores)
FPGA engine, ZedBoard at 70 MHz	**65.3**	**1.04**×	15,300 (1 engine)
FPGA engine at the routed 79 MHz	57.8	1.04×	17,300 (1 engine)
FPGA, 64-bit engine, ZCU102 at 100 MHz†	**15.9**	1.55×	62,800 (1 engine)

Table 14: FPGA against CPU on the same algorithm. CPU rows: one run on an idle machine (23 October), best of 5 passes over 2,000 in-domain inputs on one core; throughput from 8 independent processes on the 4 cores (8 threads). *Hand-optimised methods, added 1 October. FPGA rows: the term loop of the host-setup design (4,572 cycles), excluding the host's setup and finish and the register transfers; the CPU rows include everything. The CPU's own work outside its 128-term loop (truncation range, constants, discounting, parity) takes about 0.03 μs, so comparing loops only changes nothing; what the FPGA figures leave out is the transfer of 51 input and 10 output words over the processor bus, which has not been measured. †Whole evaluation, setup and finish on chip (Section 3.11); from the cycle count and clock, run bit-exact on the ZCU102, where JTAG cannot time it.

The analytic method writes out by hand the derivative of the characteristic function with respect to every input, in the spirit of Cui et al. [14], and accumulates the price and all nine Greeks in one pass over the 128 terms; it agrees with forward mode to 10^{-11} over 2,000 random inputs. The optimised bump-and-reprice reuses the characteristic function for the S_0, K, r, v_0 and θ bumps, so only nine of its pricings are full ones.

Why it is useful. On the Zynq-7020 the engine is faster than the general software methods: 1.2 times faster than forward mode, 1.6 times faster than adjoint differentiation with CoDiPack and 3.6 times faster than bump-and-reprice, but not faster than the best hand-written software: analytic Greeks on one idle laptop core take 20.3 μs, about 3.2 times faster than the ZedBoard's loop, and

several cores give far more throughput. The 64-bit engine on the ZCU102 reverses this, narrowly: 15.9 μs for the whole evaluation, about 1.3 times faster than the analytic method on one core, and 4.9, 6.5 and 15 times faster than forward mode, CoDiPack and bump-and-reprice. Four cores still give 2.5 times the throughput of one engine. What the engine keeps is the cost of its Greeks, 1.04 pricings against 1.5 for the best software and 5.7–7.6 for AD tools, a first-order rounding-error bound on every Greek, and energy: about 44 μJ per evaluation on the ZedBoard and 40 μJ for the 64-bit engine, from the engine's power under simulated activity (Section 3.12), against an estimated 76–177 μJ for the CPU at full throughput (158,000 evaluations per second) at its rated 12–28 W, about 2–4 times less. The CPU's power was not measured, so the energy ratio is still an estimate.

3.14 Many strikes at once (design study)

All strikes of one expiry share the COS grid, because the width of the truncation range does not involve the strike. Folding the payoff's phase into the characteristic function,

$$\Phi_k = \varphi(u_k)\, e^{-iu_k a} = \exp\bigl(C + v_0 D + iu_k(x-a)\bigr),$$

where $x - a$ is the same for every strike. So Φ_k and its derivatives are shared: one forward pass and *one reverse sweep per term serve every strike*, and each extra strike adds only its payoff coefficient and ten multiply-adds, 16 multiplies and one rotation per term against 234 multiplies and four CORDIC operations shared. This was built into the generator (`hardware/gen/chain.py`), scheduled and emulated, not yet built in hardware. On 8 strikes and 5 parameter sets every output's error is within 1.7 times that of the one-option engine.

	Zynq-7020 style (56-bit, 8 mult.)		**64-bit, 32 mult.**	
Strikes	**Cycles**	**Per strike**	**Cycles**	**Per strike**
1	4,662	4,662	1,577	1,577
2	4,907	2,454	1,707	854
4	5,436	1,359	1,829	457
8	8,701	1,088	2,089	261
16	8,767	548	2,854	178
32	12,860	402	4,892	153

Table 15: Price and nine Greeks for a chain of strikes, from the scheduler. The Zynq-7020 style rows use one iterative CORDIC rotator per few strikes; only about 2 strikes fit the Zynq-7020's logic.

Why it is useful. Sharing cuts the hardware cost of each further strike to about 7% of one option, which is what calibration (all strikes of an expiry, with their gradients) needs. The same sharing applies to software: the analytic C++ method prices and differentiates 32 strikes in 74.4 μs on one idle laptop core, 2.3 μs per strike. On a Zynq-7020 at 70 MHz two strikes take 70 μs, so the CPU remains faster; a 64-bit chain engine at the ZCU102's 100 MHz would take 48.9 μs for 32 strikes (scheduled, not built), 1.5 times faster than one core but slower than four. The FPGA's case against a CPU is the cost and accuracy of its Greeks and, if measured, energy, not speed.

3.15 Illustration: hedging on real market data

This section illustrates the Greeks in use; it is not evidence for the hardware. The hedge decisions depend on the hedging formula and the calibrated model, not on how the Greeks are computed: they are insensitive to word length (below), and a double-precision CPU implementation would trade the same lots. Within that scope, the engine's Greeks were used to hedge real trades. Data are the NSE end-of-day futures and options files for NIFTY, September 2025 to September 2026 (268 trading days). Only real market prices were used: an option quote counts only if at least 50

contracts traded that day. Each day the Heston model was calibrated to these quotes across all monthly expiries at once; the model's prices were within 0.019% of the index level in a typical day (median) and 0.073% at worst.

Each of 12 trades sells 20 lots of an at-the-money call with 0.15–0.30 years to expiry, hedges with NIFTY futures, and buys the call back when 0.10 years remain. The hedge ratio comes from the bit-accurate model of the generated engine (identical to the hardware). Profit and loss counts the premium, the buy-back, the futures, brokerage, securities transaction tax, exchange, SEBI and stamp charges, GST, slippage, and 30% income tax on profits. Four hedges were compared: none; the Black–Scholes delta at each day's implied volatility; the engine's Heston delta $\partial V/\partial S$; and the *minimum-variance* delta

$$\Delta_{\mathrm{MV}} \;=\; \frac{\partial V}{\partial S} \;+\; \frac{\rho\,\xi}{S}\,\frac{\partial V}{\partial v_0},$$

which also hedges the part of the variance move that is correlated with the index. It needs a second Greek, $\partial V/\partial v_0$, which the engine returns in the same pass.

Hedge (daily)	**Spread of P&L (std)**	**Mean P&L**	**Worst trade**
None	Rs 5.55 L	+Rs 0.74 L	−Rs 8.99 L
Black–Scholes delta	Rs 2.05 L	−Rs 0.07 L	−Rs 5.85 L
Heston delta, 56-bit engine	Rs 2.50 L	−Rs 0.24 L	−Rs 6.56 L
Heston minimum-variance delta, 56-bit engine	**Rs 1.26 L**	+Rs 0.18 L	**−Rs 2.54 L**

Table 16: Hedging 12 short NIFTY call trades, after all charges and tax (L = lakh).

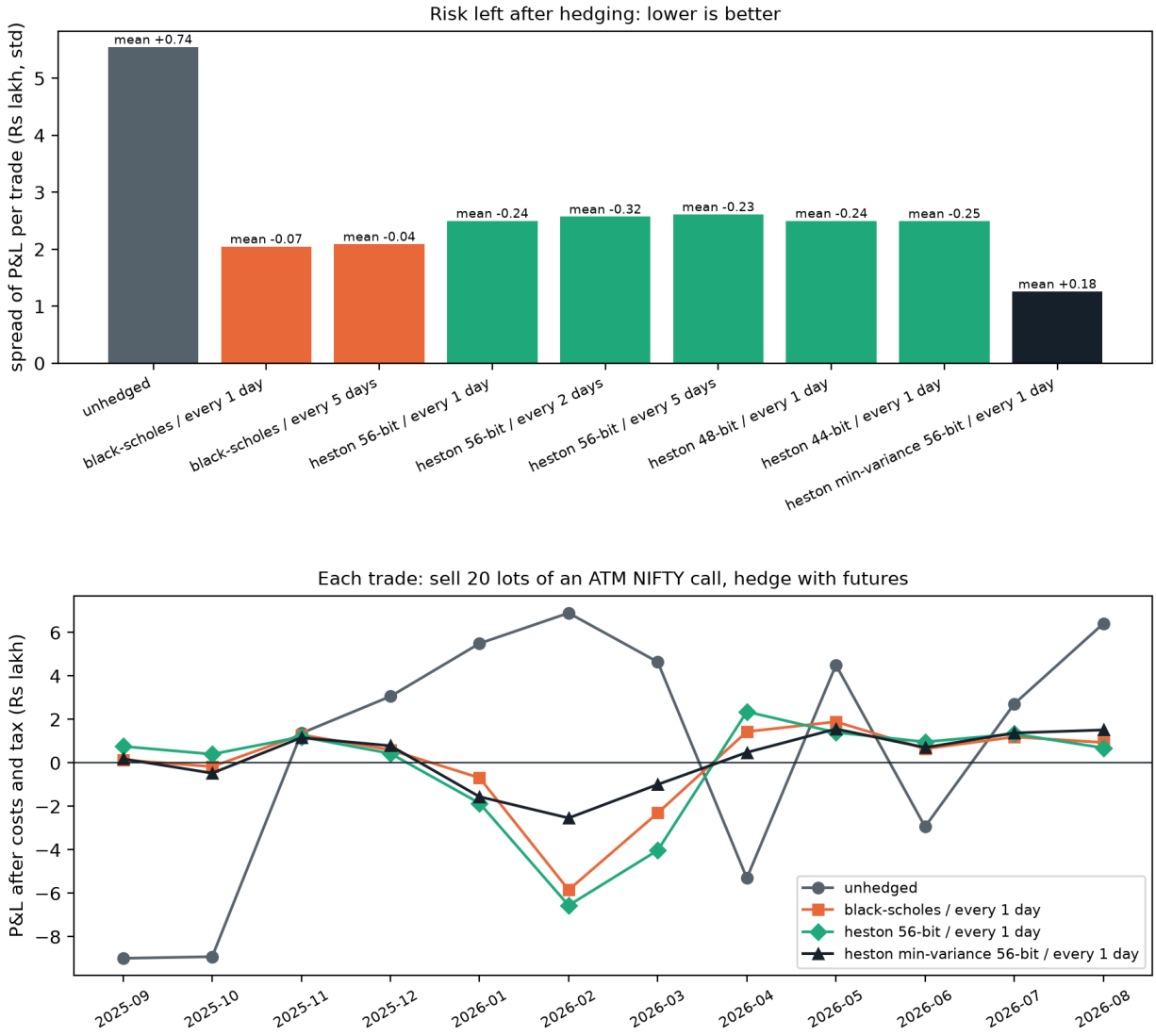


Figure 10: Top: risk left after each hedge, including rebalancing every 2 or 5 days and the 48- and 44-bit engines. Bottom: each trade's profit or loss.

The Heston delta on its own hedged worse than Black–Scholes. This is expected: it holds variance fixed, while on the index variance rises when the price falls. Adding the vega term corrects that and gave the lowest risk of all the hedges, 39% below Black–Scholes, with a worst loss of Rs 2.54 L instead of Rs 5.85 L. With 12 trades this is a case study, not a statistical result: the minimum-variance hedge lost less on 5 of the 12 trades, much of the gain comes from the worst trade, and a bootstrap gives about an 88% chance that its risk is genuinely lower. Word length did not matter: deltas from 48-bit and 44-bit engines differ from the 56-bit ones by at most 1.7×10^{-4} and 1.6×10^{-3}, below the resolution of whole futures lots; the 48-bit hedge traded exactly the same lots as the 56-bit one.

Why it is useful. The illustration shows a use for having all Greeks at once: the hedge that worked best combines two Greeks of the same option, delta and variance vega, which the engine produces together in one pass. Its benefit comes from the formula, not from the hardware, and needs more trades before it can be claimed. It also shows that for hedging, a 48-bit engine would be enough.

3.16 Improvement over the first generation

	First generation (hand-written)	**Generated engine**
Cycles, price + 9 Greeks	433,779	**4,733** (92× fewer)
Cost of the Greeks	2.16 pricings	**1.04** pricings
Advantage over bump-and-reprice	8.8×	**18.4**×
Multipliers	159 dedicated 64×64 (over 1,000 DSP)	**8 shared** (72 DSP)
Dividers	28 bit-serial per term, 80% of latency	**none**
Terms	one after another	**16 overlapping**
Fits a Zynq-7020	no	**yes**, 73% LUT
Run on silicon	no	**yes**, bit-exact
Error bound	price and Greeks, ≤ 0.21	price and Greeks, ≤ 0.103

Table 17: The first-generation engine against the generated engine.

Why it is useful. Moving from a hand-written state machine to a generated, shared, pipelined datapath is what made the design fast enough to matter and small enough to fit a low-cost FPGA.

4 Timeline

The project started on 15 August 2026; the paper is to be complete by 29 October 2026. Work up to 1 October is done; the rest is the plan (Table 18, Figure 11).

Dates	Work	Status
15–20 Aug	Black-Scholes and Heston models, software adjoint engine, MATLAB and C++ prototypes	Done
21 Aug – 9 Sep	First-generation hand-written FPGA engine and its validation	Done
10–17 Sep	Fixed-point precision fixes, error bound, bump-and-reprice baseline, the datapath generator; design fits a Zynq-7020	Done
18–24 Sep	Vivado implementation flow; project documentation	Done
25–27 Sep	AXI4-Lite interface, ZedBoard designs, first complete route (79 MHz)	Done
28–29 Sep	On silicon: one vector, then 50 cases bit-exact; put-call parity; literature check	Done
30 Sep – 1 Oct	CPU comparison; results dashboard and report	Done
1 Oct	Hand-derived analytic and optimised bump CPU baselines; strike-chain design study (§3.14)	Done
5–6 Oct	ZedBoard bitstream rebuilt with put-call parity and refitted ranges; 50-case sweep repeated, 450 / 450 bit-exact	Done
6 Oct	Forward-mode hardware baselines (§3.3); bound stated as first order and checked in 50-digit arithmetic; COS truncation error as a separate term (§3.5); held-out range sweep; claims corrected	Done
1–13 Oct	Hedging case study on real NIFTY data (first results, §3.15); next: verify charges with a broker, more trades	In progress
1 Oct	Validation sweep over 10,500 inputs including hard regimes; latency accounting (host steps, configurations)	Done
1 Oct	Shift ranges refitted at the domain edges; all designs regenerated and re-verified	Done
23 Oct	CPU timings on an idle machine	Done
21–22 Oct	ZCU104 build of the 64-bit engine (§3.11); ARM end-to-end design with the host steps in C; real NIFTY inputs as board vectors	Done
21–22 Oct	Power from simulated switching activity, both engines (§3.12)	Done
25 Oct	ZCU102 build (xczu9eg, Vivado 2026.1) and board runs: one vector, 50-case sweep, 50 NIFTY days, 1,010 / 1,010 outputs bit-exact (§3.11)	Done
Next	ZedBoard ARM end to end and NIFTY; a board power measurement and CPU package power	Needs the board
13–15 Oct	Freeze results; regenerate all figures and tables	Planned
13–22 Oct	Write the paper: introduction, related work, architecture, results	Planned
22–27 Oct	Review with the supervisor; revisions	Planned
26–28 Oct	Final literature re-check; proofreading	Planned
29 Oct	**Paper complete**	Target
After 29 Oct	Generator derives the reverse sweep automatically; a second pricing model	Next paper

Table 18: Project timeline.

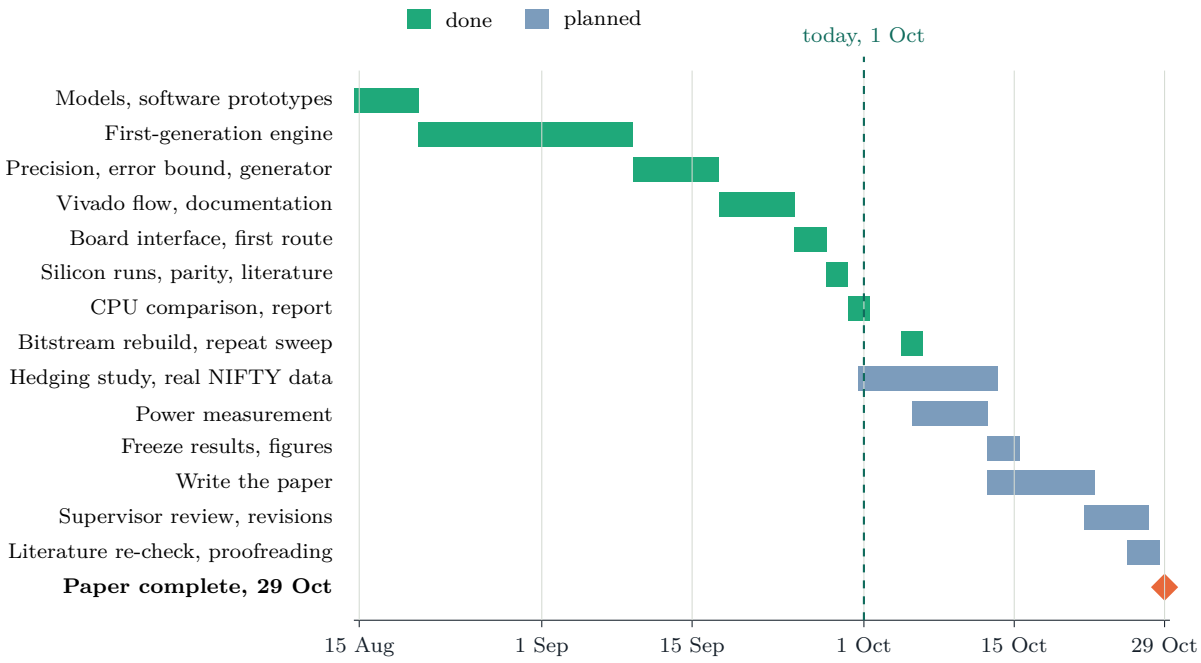


Figure 11: Timeline from project start to the paper deadline.

References

[1] S. L. Heston, "A closed-form solution for options with stochastic volatility with applications to bond and currency options," *Review of Financial Studies*, 6(2):327–343, 1993.

[2] F. Fang and C. W. Oosterlee, "A novel pricing method for European options based on Fourier-cosine series expansions," *SIAM Journal on Scientific Computing*, 31(2):826–848, 2008.

[3] M. Giles and P. Glasserman, "Smoking adjoints: fast Monte Carlo Greeks," *Risk*, January 2006.

[4] L. Capriotti, "Fast Greeks by algorithmic differentiation," *Journal of Computational Finance*, 14(3):3–35, 2011.

[5] A. Savine, *Modern Computational Finance: AAD and Parallel Simulations*, Wiley, 2018.

[6] L. Capriotti and M. Giles, "15 years of adjoint algorithmic differentiation in finance," 2024.

[7] M. Klaisoongnoen, N. Brown and O. T. Brown, "Low-power option Greeks: efficiency-driven market risk analysis using FPGAs," *HEART*, 2022.

[8] M. Klaisoongnoen et al., "Fast and energy-efficient derivatives risk analysis: streaming option Greeks on Xilinx and Intel FPGAs," *H2RC*, 2022.

[9] AMD (Xilinx), *Vitis Quantitative Finance Library*, open-source library, `https://github.com/Xilinx/Vitis_Libraries`.

[10] A. O Mahony, B. Hanzon and E. Popovici, "The role of FPGAs in modern option pricing techniques: a survey," *Electronics*, 13(16):3186, 2024.

[11] B. Zhang and C. W. Oosterlee, "Option pricing with COS method on graphics processing units," *IEEE IPDPS Workshops*, 2009.

[12] S. Linnainmaa, "Taylor expansion of the accumulated rounding error," *BIT Numerical Mathematics*, 16:146–160, 1976.

[13] M. Sagebaum, T. Albring and N. R. Gauger, "High-performance derivative computations using CoDi-Pack," *ACM Transactions on Mathematical Software*, 45(4), 2019.

[14] Y. Cui, S. del Baño Rollin and G. Germano, "Full and fast calibration of the Heston stochastic volatility model," *European Journal of Operational Research*, 263(2):625–638, 2017.